@CommerceNest hey man i am stuck in this parse cookie thing after i shifted my DB from one organisation to another kindly help me make a check list of what i should be checking in both local and prod server lets start with local first and then we can move on to prod

Got it, you're hitting the classic "Supabase cookie is garbage" problem after moving orgs. The error is literally telling you: something in your cookies is a base64/JWT string, but your code (or Supabase client) is trying to JSON.parse it.[1]

Let's be systematic. Here's the local checklist you should go through first. We'll do prod after this.

1. Kill bad cookies locally

You almost certainly have stale cookies from the old org.

- In your browser devtools → Application → Cookies:
  - Delete everything for your local domain (especially any `sb-` / `supabase-auth-token` / `supabase-session` keys).
- Restart dev server and reload.

If that makes the error temporarily disappear, the root cause is either:

- Different cookie format between old and new setup, or
- Your own code still writing something into that cookie key.[1]

2. Confirm you're not manually touching Supabase auth cookies

From the previous logs of this project, there was already a warning: "Supabase auth cookie malformed, base64 string not valid JSON. Likely caused by manual cookie writes overriding GoTrue sessions. Must let Supabase auth handle cookies unaltered."[1]

On local:

- Search the codebase for any of these:
  - `sb-`
  - `supabase-auth-token`

    - `supabaseSession`
    - `parseSupabaseCookie`
    - `JSON.parse(cookieValue)` in auth-related files
- Check:
    - `src/lib/supabaseClient.ts`
    - `src/lib/supabase.ts` / `supabase-helpers.ts`
    - Middleware: `src/middleware.ts`
    - Any custom auth routes under `src/app/**/auth/*` or `api/auth/*`.

Rules:

- You should NOT be writing Supabase's auth cookie by hand anywhere.
- Supabase client should be configured with its own storage adapter (cookies/localStorage) and left alone.
- If you find any `cookies().set(...)` or `res.cookies.set(...)` for auth tokens, that's likely your bug.[1]

If you paste the exact place where `parseSupabaseCookie` is coming from in your code, we can nuke it properly.

3. Check local env + org alignment

Since you moved DB orgs, local might still be pointing to the old project:

In `.env.local` (or wherever you load them for web):

- `NEXT_PUBLIC_SUPABASE_URL`
- `NEXT_PUBLIC_SUPABASE_ANON_KEY`

Checklist:

- These must match the new Supabase project.
- You should not have leftover `.env.local` copies in parent directories or in the old repo folder. Earlier logs show you have multiple copies / extracted trees; make sure the dev server is using the right one.[2] [3]

Quick test:

- Log `process.env.NEXT_PUBLIC_SUPABASE_URL` in `supabaseClient.ts` and ensure it's the new one when running `npm run dev`.[2]

4. Verify the Supabase client setup (local)

Open your main client file, likely `src/lib/supabaseClient.ts` or `src/lib/supabase.ts`.[2]

You want:

- A single place where Supabase client is created.

- For browser-side code:
  - Either default storage (localStorage) or the Supabase JS cookie adapter.
- For server-side (Next.js 15):
  - If you use `@supabase/auth-helpers-nextjs`, rely on their `createServerClient` and cookie helpers.
- No extra JSON.parse on raw cookie values.

If you're using the BrowserCookieAuthStorageAdapter (as the stack trace suggests), double-check you didn't change its key or wrap its value in JSON elsewhere.[1]

5. Check middleware and auth routes

Given your project already had middleware and cookie handling issues:[1]

Local checks:

- In `src/middleware.ts`:
  - Are you reading Supabase auth cookies and trying to parse them?
  - Are you overwriting any `sb-` cookie to store tenant or anything else?
- In `src/app/api/auth/**/route.ts`:
  - Ensure you're not using the same key as Supabase cookies to store random JSON (tenant, user meta, etc.).

If you are, fix it:

- Use your own names (e.g., `cn-tenant`, `cn-session`) for your stuff.
- Leave `sb-*` cookies strictly to Supabase.

6. Confirm schema/org consistency isn't breaking sessions

When you moved organisations, did you:

- Recreate the auth settings (redirect URL, site URL, allowed domains)?
- Copy RLS and schemas correctly?[1]

Locally, for the cookie error itself, the biggest risk is:

- Old JWTs encoded for the previous project hitting a client configured for the new project.

So:

- After updating env keys, restart dev server.
- Clear cookies again and perform:
  - Sign up / sign in fresh using the new project.
- Confirm the auth flow works end-to-end locally.

If it still blows up on `parseSupabaseCookie`, then the bug is 100% in your code handling cookies, not in Supabase or the org move.

7. Minimal “local OK” checklist

You’re done with local when ALL of this is true:

- [ ] No stale auth cookies: cleared and only Supabase-managed cookies get created.
- [ ] No code writes to `sb-*` / Supabase auth cookie keys.
- [ ] Supabase URL and anon key in `.env.local` match the new org project.
- [ ] A single, clean `supabaseClient` setup; no duplicate clients with different env.
- [ ] Middleware and API routes use their own cookie names and never JSON.parse Supabase’s raw cookie values.
- [ ] Fresh login in local works without “Failed to parse cookie string” in console.

Do these, and if you’re still seeing the error, send:

- The exact `supabaseClient.ts` file.
- Any custom auth helper where `parseSupabaseCookie` appears.
- A screenshot or list of the cookies in devtools after a fresh login.

Then we’ll do the prod checklist and make sure you don’t ship this broken state.

⁂

okay and what if there is a code where the authentication cookie is being written mannually what changes need to be done?

Yeah, if you’re manually writing the Supabase auth cookie, that’s the root of the mess. You need to stop doing that and separate “Supabase’s own cookie” from “your own app cookies.”[7] [8]

Here’s what needs to change.

Core rule

- Supabase’s auth cookies (`sb-*-auth-token`, etc.) must be fully managed by Supabase (or its official helpers).[9] [10]
- Your code should NEVER:
  - `JSON.stringify` or `JSON.parse` those values.
  - Rename or override those cookie keys.
  - Store extra JSON in those same keys.[7]

If you need your own cookies, use your own names (e.g. `cn_session`, `cn_tenant`, `cn_user_meta`) and treat them as pure JSON or plain strings.

What to refactor in your code

Anywhere you currently do something like:

- Writing Supabase tokens manually:
  - Bad:
    - `cookies().set('sb-xxx-auth-token', JSON.stringify(session))`
    - `cookies().set('sb-xxx-auth-token', accessToken)` when you're not using the official adapter.[7]
- Parsing Supabase cookies:
  - Bad:
    - `const data = JSON.parse(cookies().get('sb-xxx-auth-token')?.value ?? '')`
    - Doing any custom decode of that cookie value.[9] [7]

Replace with one of these patterns:

1. Use official helpers for server-side auth (recommended with Next App Router):

- Migrate to `@supabase/ssr` / server-side guide, and let it handle cookies.[11] [12] [13]
- Example pattern:
  - Create server client with `createServerClient({ cookies })`.
  - Never touch the internal auth cookie; just call `supabase.auth.getSession()` and `supabase.auth.signOut()`.[12] [14]

2. If you want your own app-level cookie:

- Read the session from Supabase client:
  - `const { data: { session } } = await supabase.auth.getSession();`
- Then write YOUR cookie with YOUR key:
  - `cookies().set('cn_session', JSON.stringify({ userId: session.user.id, role: 'admin' }), { httpOnly: true, secure: true, path: '/', sameSite: 'lax' });`[10] [9]
- When reading:
  - `const raw = cookies().get('cn_session')?.value;`
  - `const cnSession = raw ? JSON.parse(raw) : null;`

This way:

- Supabase cookies stay in their opaque JWT/base64 format that their code expects.[9] [7]
- Your JSON logic only ever touches your own cookie keys.

Concrete checklist for you

You should:

- [ ] Search the repo for `sb-` and `auth-token` and remove any manual `set` / `JSON.parse` around those.[15] [16]
- [ ] Search for `parseSupabaseCookie` and confirm it's only inside node_modules or helper libs, not your own code.[7]
- [ ] Wherever you're currently "storing session in a cookie", rename that cookie to something like `cn_session` and keep its format 100% under your control.
- [ ] After refactor, clear cookies in the browser and do a fresh login.

If you paste one concrete snippet where you're writing the cookie manually, can rewrite it line-by-line into the "correct" version for your setup (SSR, SPA, or mixed).

⁂

okay so in my code there is no manual supabase cookies now what next should i check i have the updated env for local and for vercel we'll get there after we resolve local so yeah what should i check next

Good, then stop assuming it's "some random Supabase bug." If cookies aren't written manually, the error is almost always about how the client is configured or how you're mixing environments.[29] [30]

Here's what you should check next, in order.

1. Inspect the actual cookie in the browser

You need to see what value `parseSupabaseCookie` is choking on.

- In DevTools → Application → Cookies:
  - Find any `sb-...` or `supabase-auth-token` cookie.
  - Check:
    - Is it a JSON string? Then it's wrong – Supabase expects a JWT / base64-ish token, not JSON.[31]
    - Do you see multiple auth cookies with different names or values for the same domain? That hints at mixed configs.[32]
- If it looks wrong, clear all cookies for localhost again and reload, then log in and re-check after login.

If after a fresh login that cookie is still "JSON-looking", then you've missed some code path that still writes it.

2. Verify your supabase client setup file

Open `src/lib/supabaseClient.ts` (and any `supabase.ts` / `supabase-helpers.ts`).[33]

You're looking for:

- How the browser client is created:
  - Are you using `createBrowserClient` / `createClient` from `@supabase/supabase-js`?
  - Are you passing any custom `auth.storage` or `auth.storageKey`?[34]
- Red flags:
  - A custom storage adapter that wraps values in JSON.
  - Using `BrowserCookieAuthStorageAdapter` but also having auth helpers that already manage cookies (double-management).[30] [35]

If you're using both:

- `@supabase/ssr` or the old `@supabase/auth-helpers-nextjs` on the server AND
- A cookie-based adapter on the browser,

you can end up with incompatible cookie formats using the same key. In that case, simplify:

- For browser: use default localStorage storage (`createClient` without custom `auth.storage`).[34]
- For server (Next.js app router): use a server client with cookies passed in (`createServerClient`).[36] [37]

3. Check for mixed old/new helper packages

Given the age of the project, you might be mixing packages.[38] [39]

Check `package.json`:

- Are you using ANY of:
  - `@supabase/auth-helpers-nextjs`
  - `@supabase/auth-helpers-react`
  - `@supabase/ssr`
- If yes, make sure you're not using both generations at once:
  - Either you're on the newer `@supabase/ssr` flow, or you're still on the old auth-helpers, not both.[35]

If you're on the new SSR package:

- Follow its cookie/config guide exactly (cookie prefix, key names, and options).[35] [36]

If you're on the old helpers:

- Ensure you haven't pulled some code from SSR docs and mixed them; they use different cookie strategies.[35]

4. Check middleware and app-router hooks

In `src/middleware.ts` and `src/hooks/useCustomerAuth.ts`:[33] [38]

- Look for:
  - `cookies()` + `supabase.createServerClient` usage.
  - Any call that reads auth cookies and does its own parsing / decoding.
- If middleware calls Supabase AND you are also using client-side cookie storage, they must share the same cookie config (`global` auth settings, same `storageKey`).[37] [31]

If middleware is not actually needed for auth, consider:

- Letting Supabase SSR or auth-helpers handle auth.
- Using middleware only for tenant routing, not for auth cookie manipulation.

5. Confirm you're not mixing two different Supabase projects locally

You said envs are updated, but be ruthless here:

- Log `process.env.NEXT_PUBLIC_SUPABASE_URL` and `NEXTPUBLIC_SUPABASE_ANON_KEY` in:
  - `supabaseClient.ts`
  - Any `supabaseAdmin.ts` / server client.[33]
- Make sure both point to the SAME new project locally.

If any server route or middleware is still instantiated with old env names or hard-coded URL/key, you'll get:

- Old-project JWTs stored as cookies.
- New client trying to parse them → formats mismatch → "base64 is not valid JSON" error from `parseSupabaseCookie`.[31] [32]

6. Minimal "next debugging step" for you

Here's what you should do right now, concretely:

- Step 1: Screenshot the `sb-*` cookie value after a fresh login and tell me:
  - Does it look like a long JWT-ish string (three dot-separated chunks)?
  - Or does it look like JSON `{...}`?
- Step 2: Paste the complete content of:
  - `src/lib/supabaseClient.ts`
  - Any `supabase-helpers.ts` / `supabase.ts` file.
- Step 3: Tell me which auth helper package you're using from `package.json`.

With those three, it's possible to pin down the bug instead of guessing. Right now, if you keep assuming "no manual cookie writes so it should just work," you're ignoring the other two realistic culprits: misconfigured storage adapter or mixed packages/environments.

⁂

can you explain in a layman language of why i am facing this issue and i let any ai to detect this problem i mean the root of the problem in my directory file what i have to command it

You're basically asking: "What the hell is this cookie error in simple words, and how do I point an AI at my repo to find the exact bad code?"

Here's the blunt version.

What's actually going wrong (layman terms)

- Supabase stores login info in a browser cookie in its own special format (a long encoded token, not human-readable).[42] [43]
- Somewhere in your app, some code (Supabase library or your code) is doing:
  - "Let me read that cookie and run `JSON.parse` on it because I expect it to be JSON."
- But the cookie is NOT JSON, it's that encoded token string, so `JSON.parse` blows up with:
  - "Unexpected token 'b', 'base64-...' is not valid JSON."

So the problem is not "cookies in general," it's:

- A mismatch between:
  - How the cookie is being written (encoded token), and
  - How some code is trying to read it (as JSON).[44]

That mismatch can be caused by:

- Wrong Supabase client config (using a storage adapter that expects JSON but gets a token).[43]
- Mixing old and new Supabase auth helpers that use different cookie formats.[45]
- Some part of your code still reading that cookie and parsing it as JSON.

You switched DB orgs, so now:

- Old cookies from the previous project + new client config = even more mismatch and explosions.

How to ask an AI to find the root cause in your repo

You want the AI to stop hand-waving and actually scan your files for the real culprit. That means you have to give it a very concrete, code-search style command.

Use instructions like this (copy-pasteable prompt style):

1. First: locate all Supabase client / auth helpers

- “Scan the repo and list all files that:
  - import from `@supabase/supabase-js`, `@supabase/ssr`, or `@supabase/auth-helpers-nextjs`
  - contain `createClient`, `createBrowserClient`, `createServerClient`, or `BrowserCookieAuthStorageAdapter`.
    For each file, show the code that creates the Supabase client and any `auth` configuration.”[46] [43]

2. Second: find any code that touches auth cookies

- “Search the whole project for:
  - `sb-` (cookie names)
  - `auth-token`
  - `parseSupabaseCookie`
  - `JSON.parse(` in files that also use Supabase.
    For each match, show the surrounding 20–30 lines and explain whether it is reading/writing Supabase auth cookies.”[47] [44]

3. Third: check for mixed Supabase auth packages

- “Inspect package.json and tell me:
  - Which Supabase auth-related packages I use (`@supabase/ssr`, `@supabase/auth-helpers-nextjs`, etc.).
  - Whether I’m mixing old auth-helpers and the new SSR approach.
    Then show where each of those is used in the codebase.”[48] [45]

4. Fourth: verify env usage and project mismatch

- “Find every place that reads:
  - `NEXT_PUBLIC_SUPABASE_URL`
  - `NEXT_PUBLIC_SUPABASE_ANON_KEY`
  - `SUPABASE_SERVICE_ROLE_KEY`
    Show me how many different Supabase clients are created and confirm they all use the same env vars (no hard-coded URLs or old project keys).”[49] [48]

5. Finally: ask it to pinpoint the exact root cause

- “Given all these findings, tell me:
  - Which specific file/function is reading a Supabase cookie as JSON or using an incompatible storage adapter.
  - Whether the fix is:
    - Removing JSON parsing,
    - Switching to a different storage strategy (localStorage vs cookie),
    - Or unifying to one Supabase auth helper package and cookie format.
      Then propose the exact code changes needed.”[42] [44]

If the AI can’t answer those after scanning the repo, it’s useless. You’re not asking it to “guess the bug,” you’re telling it to:

- Search for concrete patterns,
- Show you the code,
- And then reason about why that code would cause “parse cookie” to fail.

Use those exact bullets as your checklist prompt, and don’t accept a hand-wavy answer that doesn’t reference specific file paths and code snippets.

⁂

make me a prompt

Here you go – copy this and feed it to whatever repo-aware AI you’re using:

You are a senior TypeScript/Next.js/Supabase engineer. You have full access to my codebase.

I am getting this runtime error in the browser console:

“Failed to parse cookie string: SyntaxError: Unexpected token 'b', 'base64- eyJ...' is not valid JSON”
Stack mentions `parseSupabaseCookie`, `BrowserCookieAuthStorageAdapter.getItem`, and `GoTrueClient.__loadSession`.

Your job is to find the REAL root cause in this repo, not guess.

Do the following steps, in order, and reference concrete file paths and code snippets for every claim:

1. Locate all Supabase client setups

- Search for imports from:
  - `@supabase/supabase-js`
  - `@supabase/ssr`
  - `@supabase/auth-helpers-nextjs`
- For each file (e.g. `src/lib/supabaseClient.ts`, `supabase.ts`, `supabase-helpers.ts`, server utilities, middleware):
  - Show the code where the Supabase client is created.
  - Show any `auth` config: `storage`, `storageKey`, `flowType`, `BrowserCookieAuthStorageAdapter`, etc.
  - Tell me whether it uses localStorage or cookies for auth.

2. Find all code that reads or writes Supabase auth cookies

- Search the entire repo for:
  - `sb-`
  - `auth-token`
  - `parseSupabaseCookie`

    - `supabase-auth`
    - `JSON.parse(` in files that also import Supabase.
- For every match:
    - Show the surrounding ~20–30 lines.
    - Tell me whether this code is:
        - Reading a Supabase auth cookie and parsing it as JSON.
        - Writing to a Supabase auth cookie key.
        - Overriding the cookie name or format.

3. Check for mixed or conflicting Supabase auth packages

- Open `package.json` and list all Supabase-related packages, especially:
    - `@supabase/supabase-js`
    - `@supabase/ssr`
    - `@supabase/auth-helpers-nextjs` / `@supabase/auth-helpers-react`
- Tell me:
    - Which auth approach the project is actually using (old auth-helpers vs new SSR).
    - Where each package is used in the codebase.
    - Whether I'm mixing old and new helpers or using an auth helper AND a cookie storage adapter in a conflicting way.

4. Verify environment variable usage and project consistency

- Search for all usages of:
    - `NEXT_PUBLIC_SUPABASE_URL`
    - `NEXT_PUBLIC_SUPABASE_ANON_KEY`
    - `SUPABASE_SERVICE_ROLE_KEY`
- For each Supabase client instantiation:
    - Show the exact env vars used.
    - Confirm whether ALL clients (browser, server, admin, middleware) are using the same project URL and keys (no hard-coded or legacy values).

5. Diagnose the exact cause of the parseSupabaseCookie error
   Based on the findings above, answer these explicitly:

- Which specific file/function is ultimately causing `parseSupabaseCookie` to receive a non-JSON value but still call `JSON.parse`?
- Is the root cause:
    - A custom storage adapter wrapping values as JSON?
    - A manual read/parse of Supabase's auth cookie?
    - Mixed auth-helper packages with incompatible cookie formats?
    - Different Supabase projects/keys being used in different parts of the app?

6. Propose concrete, minimal fixes

- Point to the exact lines that need to change.
- For each change, show before/after code.
- Explain in plain language:
  - How the fix stops `parseSupabaseCookie` from failing.
  - Whether I should switch to localStorage auth, standard cookie auth, or a single SSR/auth-helper approach.
  - Any cleanup I must do (e.g. clearing old cookies, removing duplicate clients, unifying env vars).

Do not give vague advice. I want file paths, code snippets, and a clear "this is the line that's wrong and here is the corrected version."

this is the summary i got

SUMMARY
Root Cause: Mixing @supabase/auth-helpers-nextjs (deprecated) with @supabase/ssr (modern) creates incompatible cookie formats.

Files to Change:

src/components/admin/AuthGate.tsx - Use supabaseClient instead of createClientComponentClient
src/app/api/auth/signout/route.ts - Use createServerClient from @supabase/ssr
package.json - Remove @supabase/auth-helpers-nextjs
src/lib/supabaseClient.ts - Add one-time cookie cleanup
After Fix:

All code uses @supabase/ssr consistently
No more cookie parsing errors
Users need to log in once after clearing old cookies
This is a code architecture issue, not a database migration issue. The solution is to standardize on one auth package (@supabase/ssr).

Good, that summary is actually solid. You were mixing old and new Supabase auth stacks, which is exactly the kind of architectural mess that causes this cookie nonsense.[50] [51]

Here's what you need to do next, no fluff.

1. Fix package.json (stop mixing helpers)

- Remove `@supabase/auth-helpers-nextjs` from dependencies.
- Ensure you have `@supabase/ssr` + `@supabase/supabase-js` only.[52] [50]

Then:

- `npm install` (or `pnpm/yarn`).

- Restart dev server after this.

If you don't actually remove the old package, you're just papering over the bug.

2. Update AuthGate to use your central client

In `src/components/admin/AuthGate.tsx`:

- Stop using `createClientComponentClient` or anything imported from `@supabase/auth-helpers-nextjs`.[50]
- Instead:
  - Import your existing browser client from `src/lib/supabaseClient.ts`.
  - Use that single client everywhere in the UI.

Example shape (adapt to your file):

- Bad:
  - `const supabase = createClientComponentClient<Database>();`
- Good:
  - `import { supabaseClient } from '@/lib/supabaseClient';`
  - `const supabase = supabaseClient;`

One app, one browser Supabase client. Anything else is asking for bugs.

3. Fix signout route to use @supabase/ssr

In `src/app/api/auth/signout/route.ts`:

- Remove any `createRouteHandlerClient` / old helper imports.[50]
- Use `createServerClient` from `@supabase/ssr` and the Next.js cookies API.[53] [52]

Shape (pseudo):

- `import { createServerClient } from '@supabase/ssr';`
- `import { cookies } from 'next/headers';`
- Inside handler:
  - `const cookieStore = cookies();`
  - `const supabase = createServerClient(process.env.NEXT_PUBLIC_SUPABASE_URL!, process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!, { cookies: { get: ..., set: ..., remove: ... } });`
  - Call `supabase.auth.signOut()` and return response.

Follow the official App Router + SSR guide for exact cookie adapter code.[52] [53]

4. Optional but smart: one-time cookie cleanup

In `src/lib/supabaseClient.ts` (or a small init util):

- Before creating the client, you can nuke any legacy auth-helper cookies whose names don't match `@supabase/ssr`'s pattern.[54] [50]
- Even a simple "clear all `sb-` cookies once" on load is acceptable during this migration, as long as you understand it will sign users out and they'll log in again.

But don't overcomplicate this; the main fix is not the cleanup, it's unifying the auth stack.

5. Your local testing checklist

After doing the above:

- Clear site cookies in the browser (one time).
- `npm run dev`.
- Hit your app:
  - Login flow should work.
  - Check console: the "parse cookie string" error should be gone.
- Verify:
  - Admin pages behind `AuthGate` still work.
  - Signout endpoint works and actually clears auth.

If any error remains, send:

- The updated `supabaseClient.ts`.
- The updated `AuthGate.tsx`.
- The updated `signout/route.ts`.

Right now, the summary nailed it: this was never about "DB moved orgs", it was about you running two Supabase auth systems side-by-side like a Frankenstein setup. Clean it up, standardize on `@supabase/ssr`, and the cookie error dies.

⁂

make me a prompt

Here's the next prompt you should give the repo-aware AI, based on that summary:

You already found that the root cause is mixing `@supabase/auth-helpers-nextjs` (deprecated) with `@supabase/ssr` (modern), which creates incompatible auth cookie formats and leads to `parseSupabaseCookie` errors.

Now I want you to generate exact, copy-pasteable fixes for my project.

Do the following, step by step, and always include:

- Full file paths.
- Before/after code blocks.
- Any imports that need to be added or removed.

1. Clean up package.json

- Open `package.json`.
- Remove `@supabase/auth-helpers-nextjs` (and any other deprecated Supabase auth helper packages).
- Ensure `@supabase/ssr` and `@supabase/supabase-js` are present with appropriate versions.
- Output the updated `"dependencies"` section so I can copy it.

2. Fix src

make me a prompt for Kiro as he is the one who gave me that summary

Here's a direct prompt you can give Kiro:

Kiro, thanks for the summary. Now I need you to go all the way and give me concrete, copy-pasteable fixes.

Context:
You identified the root cause as mixing `@supabase/auth-helpers-nextjs` with `@supabase/ssr`, which leads to incompatible cookie formats and the `parseSupabaseCookie` / "base64... is not valid JSON" errors.

I want you to:

1. Package.json cleanup

- Open `package.json` and:
  - Remove `@supabase/auth-helpers-nextjs` and any other deprecated Supabase auth helper packages.
  - Ensure `@supabase/ssr` and `@supabase/supabase-js` are correctly declared.
- Show me the updated `"dependencies"` block I should use.

2. src/components/admin/AuthGate.tsx

- Open this file and:
  - Replace any usage of `createClientComponentClient` or other imports from `@supabase/auth-helpers-nextjs` with usage of my central Supabase browser client from `src/lib/supabaseClient.ts`.
  - Provide a before/after diff:
    - What the imports and Supabase initialization look like now.
    - What they should look like after the fix.

- The result should be: one shared browser client, no auth-helper imports here.

3. src/app/api/auth/signout/route.ts

- Open this file and:
  - Remove old `@supabase/auth-helpers-nextjs` usage (e.g. `createRouteHandlerClient`).
  - Implement signout using `createServerClient` from `@supabase/ssr` and `cookies` from `next/headers`, following best practices.
- Give me complete, final code for this route (including imports) that I can paste in.

4. src/lib/supabaseClient.ts

- Review this file and:
  - Confirm it is compatible with `@supabase/ssr` (no leftover auth-helper logic).
  - If helpful, add a one-time "legacy cookie cleanup" function that clears old auth-helper cookies on the client to avoid leftover bad cookies.
- Show the final version of this file, or at least the full client creation code plus any cleanup logic.

5. Final verification checklist

- Based on the above changes, give me a short checklist of what I must do locally to verify the fix:
  - Commands to run (e.g. `npm install`, `npm run dev`).
  - Browser steps (clear cookies, re-login, etc.).
  - What I should see/not see in the console.

Be specific. I don't want high-level advice, I want exact code blocks and file paths that I can copy-paste into my project.

no i meant it has the access of my directory so it can automatically apply the change needed

Got it, you want Kiro to not just suggest code, but actually modify your repo automatically.

Use this prompt:

Kiro, you have full access to my project directory. I want you to automatically apply the auth fix you summarized (stop mixing `@supabase/auth-helpers-nextjs` with `@supabase/ssr`) so that the Supabase cookie parsing error goes away.

Do the following, and actually edit the files in place:

1. package.json

- Open package.json.
- Remove `@supabase/auth-helpers-nextjs` (and any other deprecated Supabase auth helper packages).

- Ensure `@supabase/ssr` and `@supabase/supabase-js` are present with appropriate, compatible versions.
- Save the file.

2. src/components/admin/AuthGate.tsx

- Open this file.
- Replace any imports/usages of `createClientComponentClient` or anything from `@supabase/auth-helpers-nextjs` with the central browser client from `src/lib/supabaseClient.ts`.
- Result: AuthGate should import and use a single shared Supabase client, with no auth-helper imports.
- Save the file.

3. src/app/api/auth/signout/route.ts

- Open this file.
- Remove any use of `createRouteHandlerClient` or other `@supabase/auth-helpers-nextjs` APIs.
- Re-implement the handler using:
  - `createServerClient` from `@supabase/ssr`
  - `cookies` from `next/headers`
  - The same Supabase URL and anon key env vars as the rest of the project.
- Make sure signout clears the auth cookies correctly and returns a valid Next.js Response.
- Save the file.

4. src/lib/supabaseClient.ts

- Open this file.
- Ensure it is the single source of truth for the browser Supabase client, using `@supabase/supabase-js` only (no auth-helper imports here).
- If needed, add a small one-time client-side effect that clears any legacy auth-helper cookies whose names conflict with the new `@supabase/ssr` cookies, so old sessions don't break parsing.
- Save the file.

5. Consistency check

- Search the whole project for `@supabase/auth-helpers-nextjs`.
- Remove or refactor any remaining usages to use either:
  - The browser client from `supabaseClient.ts`, or
  - A server client created via `@supabase/ssr`.
- Make sure there is exactly one auth stack: `@supabase/ssr` + `@supabase/supabase-js`.

6. Report back
When you're done applying changes, tell me:

- Which files you modified.

- A brief summary of the main edits.
- Any manual steps I must run:
  - e.g. `npm install`, `npm run dev`, clearing browser cookies, re-login, etc.

Do not just describe the changes—actually edit the files in my repo and then summarize what you did.

this is what i get just need to test

Before: Mixed @supabase/auth-helpers-nextjs (deprecated) + @supabase/ssr (modern) = incompatible cookie formats = parse errors

After: Single auth stack using only @supabase/ssr = consistent cookie format = no parse errors

Result:

✅ No more "Failed to parse cookie string" errors
✅ Consistent authentication across all components
✅ Using modern, supported Supabase packages
✅ Automatic cleanup of legacy cookies
The cookie parsing error is now permanently fixed!

Nice. That's exactly what you wanted: one auth stack, one cookie format, zero bullshit.

Here's what you should do now, step by step, to actually verify it and be sure it's not lying to you:

1. Clean and restart locally

- Run:
  - `npm install` (if dependencies changed).
  - Optional but smart: `rm -rf .next` then `npm run dev`.
- This ensures you're not seeing cached behavior.

2. Hard reset browser side

- In your browser:
  - Clear cookies for your local dev domain (localhost / 127.0.0.1).
  - Completely close all dev tabs for that app.
  - Open a fresh tab and hit your local URL.

3. Test the full auth flow

- Go to:
  - Admin login.
  - Customer login (if separate).
- Actions to test:
  - Sign in.
  - Navigate to pages that previously triggered the error.
  - Sign out (hit the signout route you changed).
- Watch the console:
  - There should be NO “Failed to parse cookie string” / `parseSupabaseCookie` / `base64 is not valid JSON` messages.

If you still see any cookie parse error:

- Grab:
  - The exact console error + stack.
  - Current `sb-*` cookie values from DevTools.
- Then it means there’s still some leftover path using the old helpers, and we’ll hunt that last one down.

4. Quick sanity checklist

You’re done for local when:

- [ ] `package.json` has NO `@supabase/auth-helpers-nextjs`.
- [ ] Global search for `"auth-helpers-nextjs"` returns zero hits.
- [ ] Only `@supabase/ssr` + `@supabase/supabase-js` are used for auth.
- [ ] Login + logout + protected pages all work.
- [ ] Console stays clean of cookie parse errors.

Run those checks properly. If it passes, then we move on to production envs next and make sure you don’t reintroduce this crap there.

now lets move to prod server but before that need to push all the changes in to github and then we can proceed what you say ?

Yes, that’s exactly what you should do next. But don’t do it sloppily.

Here’s the minimal, sane workflow before we even touch prod.

1. Commit properly

From your project root:

- Check what changed:
  - `git status`
- Review diffs for the key files (auth stuff, package.json, etc.).
  - Make sure there's no trash like `.env.local` or build artifacts staged.

Then:

- Stage only real source changes:
  - `git add src lib package.json package-lock.json` (adjust paths as needed).
- Commit with a clear message, e.g.:
  - `git commit -m "Refactor Supabase auth to use @supabase/ssr only"`

If `git status` shows `.env.local` or other secrets staged, unstage them or update `.gitignore`. Never push secrets.[55] [56]

2. Sync with remote before pushing

Don't be lazy here:

- `git pull origin <your-branch>`
  Resolve conflicts if any.
- Then:
  - `git push origin <your-branch>`[56] [55]

If main/master is your prod branch, make sure this branch is actually ready before opening a PR or merging.

3. After push: then we talk prod

Once:

- Local tests pass.
- Changes are committed and pushed.

Then next step is:

- Check Vercel (or your prod platform) env vars match the new Supabase project and auth setup.[57] [58] [59]
- Trigger a preview deployment first, test auth there, then promote/merge to prod.

So yes: push to GitHub now, but do it cleanly following the steps above. When you're done and the branch is on GitHub, tell which:

- Branch name.
- Deployment flow (Vercel? something else?).

Then you'll get a production checklist just as ruthless as the local one.

⁂

now that i pushed it into github and its get build as well and i also tried to create the product and got this issue

You pushed to prod with the auth refactor half-wired, so now you've got three issues at once:

- 401 on `/api/auth/check-tenant-access`
- "Multiple GoTrueClient instances detected"
- The same cookie parse error as before (but now in prod)

That means:

- You still have more than one Supabase client in the browser.
- Prod cookies weren't cleared or are still using the old helper format.
- `check-tenant-access` is probably using a different auth stack (or wrong env) than the rest of the app.

Here's what you need to do next for PROD specifically.

1. Confirm you really killed all extra clients

In the deployed code (same repo):

- Global search for:
  - `createClientComponentClient`
  - `createBrowserClient`
  - `new GoTrueClient`
  - Any direct `new SupabaseClient(...)` calls.
- You should have:
  - One browser client (from `src/lib/supabaseClient.ts`).
  - Server clients created via `@supabase/ssr` using the cookies adapter.[75] [76]

The "Multiple GoTrueClient instances detected" warning means you still spin up more than one auth client in the same browser context, usually because:

- You create a Supabase client in a hook or component instead of a shared module.
- You still have leftover auth-helper code in some component.[77]

If you find any client creation that isn't in your central `supabaseClient.ts` or your SSR client factory, move it there.

2. Fix /api/auth/check-tenant-access

This route is returning 401 and likely still using old auth-helper logic or wrong envs.[78] [79]

Open `src/app/api/auth/check-tenant-access/route.ts` and check:

- Imports:
  - If it imports from `@supabase/auth-helpers-nextjs` → wrong. Replace with `@supabase/ssr`. [80] [81]
- Supabase client creation:
  - Must use `createServerClient` with `cookies` from `next/headers`, exactly like your signout route.
- Tenant check:
  - It should call `supabase.auth.getUser()`/`getSession()` and then verify tenant from DB. Any 401 means:
    - No session cookie was read (cookie format mismatch).
    - Or wrong Supabase project URL/key on prod.

Also check prod env on Vercel:

- `NEXT_PUBLIC_SUPABASE_URL`
- `NEXT_PUBLIC_SUPABASE_ANON_KEY`
- `SUPABASE_SERVICE_ROLE_KEY` (if used server-side)

They must match your new Supabase project, not the old org.[82] [83]

3. Deal with prod cookies

Prod browser still has old cookies from the previous auth-helper setup. You fixed code, but you didn't fix reality.

For each domain (admin, tenant, etc.):

- Clear site cookies manually in the browser (for your real domain).
- Reload and log in again.
- Check DevTools:
  - There should be a single `sb-...` cookie per project.
  - After that, the `base64-... is not valid JSON` error should drop if the client is consistent. [84] [85]

If the error only appears until you clear cookies, you can optionally add a "legacy cookie cleanup" in a small client-side effect that deletes known old cookie names once in production.

4. Connect the dots before trying product creation again

Do this sequence in prod:

1. Visit admin login page → log in.
2. Check console:
   - If you still see “Multiple GoTrueClient instances” or parse errors on the login page itself, the client setup is still wrong. Fix that first.
3. Only after login is clean, go to product creation and test.

If creating a product calls `/api/admin/...` routes that rely on Supabase sessions, they will blow up as long as:

- `/api/auth/check-tenant-access` is failing 401.
- Or the server can’t read the cookie due to mismatch between `@supabase/ssr` config and the cookie format.[76] [81]

5. What I want from you next

Don’t just say “it’s still broken.” Get me something I can use:

- Paste:
  - The current code of `src/app/api/auth/check-tenant-access/route.ts`.
  - The current code of `src/lib/supabaseClient.ts`.
- Tell me:
  - After clearing cookies on the live domain and logging in again, do you still see:
    - “Multiple GoTrueClient instances detected”?
    - “Failed to parse cookie string”?

Right now, you fixed local but pushed without finishing the prod side: route still on old auth stack, multiple clients, and stale cookies. Clean those three, and the product creation error will disappear with it.

⁂

now for the code part make a prompt for kiro and for the vercerl part ill be doing it but first start with code

Here’s a focused prompt you can give Kiro to fix the code side for prod:

Kiro, focus on CODE ONLY for now (I’ll handle Vercel env vars myself).

Goal: Fully standardize Supabase auth on `@supabase/ssr` and eliminate:

- “Multiple GoTrueClient instances detected”

- “Failed to parse cookie string: base64-... is not valid JSON”
- 401s from `/api/auth/check-tenant-access`

You have full access to the repo. Please directly edit files.

Do the following:

1. Ensure a single browser Supabase client

- Open `src/lib/supabaseClient.ts`.
- Make this the ONLY place where the browser Supabase client is created using `@supabase/supabase-js`.
- Make sure:
  - No `@supabase/auth-helpers-nextjs` imports remain.
  - Client is exported and reused by components (not recreated in hooks).
- Then search the entire project for any other Supabase client creation in the browser:
  - `createClientComponentClient`
  - `createBrowserClient`
  - `new GoTrueClient`
  - `new SupabaseClient`
- Refactor those to use the shared client from `supabaseClient.ts` instead.
- Save all changes.

2. Fix `/api/auth/check-tenant-access` to use @supabase/ssr

- Open `src/app/api/auth/check-tenant-access/route.ts`.
- Remove any use of `@supabase/auth-helpers-nextjs` (e.g. `createRouteHandlerClient`).
- Implement the handler using:
  - `createServerClient` from `@supabase/ssr`
  - `cookies` from `next/headers`
- It should:
  - Read the Supabase session from cookies.
  - Return 401 if there is no valid user.
  - Perform the tenant access check using the same Supabase project (URL/key) as the rest of the app.
- Ensure the cookie adapter code matches the current recommended pattern from Supabase SSR docs.
- Save the file.

3. Remove any remaining auth-helper usage

- Global search for `@supabase/auth-helpers-nextjs` or `createClientComponentClient` or `createRouteHandlerClient`.

- For each occurrence:
  - Refactor to use either:
    - The browser client from `src/lib/supabaseClient.ts` (for client components), or
    - A server client built via `createServerClient` from `@supabase/ssr` (for route handlers / server components).
- The end result should be: one auth stack (`@supabase/ssr` + `@supabase/supabase-js`), no mixed helpers.

4. Optional: legacy cookie cleanup helper

- If appropriate, add a small client-side util (e.g. in `src/lib/supabaseClient.ts` or a tiny hook) that:
  - On first load in the browser, deletes any old auth-helper cookie names that would conflict with the new cookies.
- This should be safe and minimal, just enough to prevent old cookie formats from triggering `parseSupabaseCookie`.

5. Consistency check and report

- After edits:
  - Confirm that there is exactly one browser Supabase client.
  - Confirm all API auth routes use `@supabase/ssr` with the same env vars.
- Then summarize:
  - Which files you changed (with paths).
  - What you changed in each (high level).
  - Any notes I must remember when I go to configure Vercel (but don't touch Vercel yourself).

Please DO NOT just describe changes—actually modify the files in the repo, then give me the summary so I can pull and deploy.

checked the environ variables and this is what i got in console

A 401 from `/api/auth/check-tenant-access` after you "fixed" env vars means one thing: the API still doesn't see a valid Supabase session for that request. The console is just proving that.[90] [91]

You need to stop assuming envs were the magic bullet. For code, here's what to tell Kiro next so he can finish the job properly.

Use this prompt:

Kiro, `/api/auth/check-tenant-access` is still returning 401 in production, even after updating environment variables. That means the route is not reading a valid Supabase session from cookies.

You already standardized on `@supabase/ssr`. Now I want you to specifically fix this route and verify the wiring end-to-end.

Tasks (edit files directly):

1. Inspect and fix src/app/api/auth/check-tenant-access/route.ts

- Open this file and:
  - Confirm it imports `createServerClient` from `@supabase/ssr` and `cookies` from `next/headers`.
  - Confirm it uses the same env vars as the rest of the app:
    - `process.env.NEXT_PUBLIC_SUPABASE_URL`
    - `process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY`
- Implement the standard SSR cookies adapter exactly once, like in the Supabase SSR docs, so that Supabase can read the auth cookies for this request.
- In the handler:
  - Call `supabase.auth.getUser()` (or `getSession()`) and log/handle the result.
  - If there is no user, return 401.
  - If there is a user, perform the tenant access check and return 200 with a JSON body.
- Make sure there is no extra auth-helper or second client created here.

2. Verify that the client actually sends auth cookies to this route

- Find where `/api/auth/check-tenant-access` is called on the client (likely in a hook or layout).
- Confirm it is called from a logged-in context and that:
  - The call is a normal fetch to a relative URL (so browser includes cookies automatically).
  - There is no `credentials: 'omit'` (if you set credentials, it should be `include` or `same-origin`).
- If credentials are missing, update the fetch to ensure Supabase auth cookies are sent.

3. Add temporary debug logging (server side)

- In `check-tenant-access/route.ts`, add minimal logging:
  - Log whether cookies exist and whether `supabase.auth.getUser()` returns a user or null.
- This is just to confirm in logs why it returns 401.
- Keep the logs small and safe (no tokens).

4. Consistency check

- Ensure this route uses the same SSR client creation code as your signout route and any other auth-protected API routes.
- Ensure there is no duplicate Supabase client creation in this file.

5. Report

- After editing, summarize:

- The final code of `check-tenant-access/route.ts` (or at least the full handler + imports).
- Where it is called from on the client.
- Whether it now correctly finds a user when called from an authenticated page.

Do not just explain what to do—apply the code changes in the repo, then tell me what you changed so I can deploy and test.

⁂

1. Senlysh-thread12.md
2. 02-complete-file-tree.txt
3. Senlysh-Thread3.md
4. image.jpg
5. hey-so-ill-be-sharing-some-childfiles-like-basic-i.pdf
6. 03-file-inventory.csv
7. https://github.com/orgs/supabase/discussions/30339
8. Senlysh-thread12.md
9. https://supabase.com/docs/guides/auth/sessions
10. https://supabase.com/docs/guides/auth
11. https://supabase.com/docs/guides/auth/auth-helpers/nextjs
12. https://supabase.com/docs/guides/auth/server-side/advanced-guide
13. https://supabase.com/docs/guides/troubleshooting/how-to-migrate-from-supabase-auth-helpers-to-ssr-package-5NRunM
14. https://supabase.com/docs/guides/getting-started/tutorials/with-nextjs
15. https://github.com/supabase/supabase-js/issues/963
16. 02-complete-file-tree.txt
17. https://www.reddit.com/r/reactjs/comments/13dyyv9/nextjs_and_supabase_how_to_get_user_auth/
18. https://www.youtube.com/watch?v=KmJN-bEayeY
19. https://github.com/orgs/supabase/discussions/12303
20. https://stackoverflow.com/questions/79407965/i-have-a-next-js-15-console-warning-error-route-uses-cookies-get-cooki
21. https://www.reddit.com/r/Supabase/comments/x6788z/how_do_you_auth_with_a_http_only_cookie/
22. https://github.com/supabase/ssr/issues/36
23. https://supabase.com/docs/guides/getting-started/ai-prompts/nextjs-supabase-auth
24. https://stackoverflow.com/questions/78696903/unable-to-access-authorisation-cookie
25. https://nextjs.org/docs/app/guides/authentication
26. https://github.com/supabase/gotrue-js/issues/280
27. https://www.npmjs.com/package/@supabase/gotrue-js
28. https://stackoverflow.com/questions/76029908/error-using-next-js13-and-next-supabase-auth
29. Senlysh-thread12.md
30. https://github.com/orgs/supabase/discussions/30339

31. https://supabase.com/docs/guides/auth/sessions
32. https://github.com/supabase/supabase-js/issues/963
33. 02-complete-file-tree.txt
34. https://www.npmjs.com/package/@supabase/gotrue-js
35. https://supabase.com/docs/guides/troubleshooting/how-to-migrate-from-supabase-auth-helpers-to-ssr-package-5NRunM
36. https://supabase.com/docs/guides/auth/auth-helpers/nextjs
37. https://supabase.com/docs/guides/auth/server-side/advanced-guide
38. Senlysh-Thread1.md
39. Senlysh-Thread3.md
40. 03-file-inventory.csv
41. hey-so-ill-be-sharing-some-childfiles-like-basic-i.pdf
42. https://supabase.com/docs/guides/auth/sessions
43. https://www.npmjs.com/package/@supabase/gotrue-js
44. https://github.com/orgs/supabase/discussions/30339
45. https://supabase.com/docs/guides/troubleshooting/how-to-migrate-from-supabase-auth-helpers-to-ssr-package-5NRunM
46. https://supabase.com/docs/guides/auth/auth-helpers/nextjs
47. Senlysh-thread12.md
48. Senlysh-Thread1.md
49. 02-complete-file-tree.txt
50. https://supabase.com/docs/guides/troubleshooting/how-to-migrate-from-supabase-auth-helpers-to-ssr-package-5NRunM
51. https://github.com/orgs/supabase/discussions/30339
52. https://supabase.com/docs/guides/auth/auth-helpers/nextjs
53. https://supabase.com/docs/guides/auth/server-side/advanced-guide
54. https://supabase.com/docs/guides/auth/sessions
55. https://dev.to/aneeqakhan/best-practices-for-git-and-version-control-588m
56. https://github.com/git-guides/git-push
57. https://vercel.com/docs/environment-variables
58. https://vercel.com/marketplace/supabase
59. https://supabase.com/docs/guides/deployment/managing-environments
60. https://about.gitlab.com/topics/version-control/what-are-gitlab-flow-best-practices/
61. https://supabase.com/partners/integrations/vercel
62. https://www.tczhong.com/posts/frontend/mission-impossible-deploy-nextjs-with-supabase-auth-in-firebase/
63. https://codefresh.io/learn/github-actions/deployment-with-github-actions-quick-tutorial-and-5-best-practices/
64. https://github.com/orgs/supabase/discussions/12303
65. https://spacelift.io/blog/gitops-best-practices

66. https://supabase.com/docs/guides/auth/server-side/creating-a-client
67. https://makerkit.dev/docs/next-supabase-turbo/going-to-production/checklist
68. https://supabase.com/docs/guides/auth/server-side/advanced-guide
69. https://supabase.com/docs/guides/functions/secrets
70. https://vercel.com/templates/next.js/supabase
71. https://developers.redhat.com/articles/2022/07/20/git-workflows-best-practices-gitops-deployments
72. https://stackoverflow.com/questions/9826977/best-practice-to-remote-push-in-git
73. https://www.reddit.com/r/devops/comments/vwvb23/how_can_i_safely_use_git_push_in_a_build_pipeline/
74. https://supabase.com/docs/guides/getting-started/tutorials/with-nextjs
75. https://supabase.com/docs/guides/auth/auth-helpers/nextjs
76. https://supabase.com/docs/guides/auth/server-side/creating-a-client
77. https://www.npmjs.com/package/@supabase/gotrue-js
78. 01-folder-structure.txt
79. 02-complete-file-tree.txt
80. https://supabase.com/docs/guides/troubleshooting/how-to-migrate-from-supabase-auth-helpers-to-ssr-package-5NRunM
81. https://supabase.com/docs/guides/auth/server-side/advanced-guide
82. https://supabase.com/docs/guides/deployment/managing-environments
83. https://supabase.com/docs/guides/functions/secrets
84. https://github.com/orgs/supabase/discussions/12303
85. https://supabase.com/docs/guides/auth/sessions
86. image.jpg
87. Senlysh-Thread6.md
88. Senlysh-Thread1.md
89. Senlysh-Thread13.md
90. https://supabase.com/docs/guides/auth/server-side/advanced-guide
91. https://supabase.com/docs/guides/auth/server-side/creating-a-client
92. image.jpg